

CASE STUDY
CIA-3 – Component 1
CSEAM731 – REINFORCEMENT LEARNING

**Adaptive Traffic Signal Control at an Isolated Urban
Intersection:
A Reinforcement Learning Formulation using Q-Learning**

Submitted by

Niharika Singh – 2362121
Esha Bobby – 2362067
Aarnav Sabherwal – 2362001
Amrit Raj Dhar – 2362030

Class: 7BTCS AIML – C
Course: Reinforcement Learning
Institution: CHRIST (Deemed to be University)
Academic Year: 2026–2027

Abstract

This case study formulates adaptive traffic signal control at an isolated four-approach urban intersection as a sequential decision-making problem suitable for reinforcement learning (RL). The controller observes discretised approach queues, the active signal phase and a minimum-green timer, and must decide, at every simulation tick, whether to keep the current phase or request a switch. A custom Gym-style simulator, TrafficSignalEnv, models stochastic Poisson vehicle arrivals, a directional demand drift within each peak-period episode, and occasional discharge stalls, giving rise to a finite, discrete Markov Decision Process (MDP) with 6,480 states and two actions. The case study defines the problem, the RL environment, the agent, the reward structure and the RL-relevant characteristics of the task, and identifies Q-learning as the most suitable tabular temporal-difference control algorithm for this setting because it is off-policy, model-free, and yields a greedy policy that is directly usable for real-time deployment. As an innovation extension, the case study proposes Safe Reinforcement Learning through a hard action shield that enforces minimum-green, clearance and spillback-prevention constraints independently of the learned policy. This formulation provides the conceptual foundation for the empirical implementation and evaluation to be carried out separately.

Keywords: *Adaptive Traffic Signal Control, Reinforcement Learning, Markov Decision Process, Q-learning, Sequential Decision-Making, Safe RL*

1. Introduction

1.1 Background

Urban signalised intersections are bottlenecks in surface transportation. When a signal plan is poorly matched to demand, queues accumulate on upstream approaches, spillback can block neighbouring junctions, and public-transport headways can be disrupted. Conventional practice commonly uses fixed-time plans, time-of-day schedules and actuated rules based on current vehicle presence. These controllers are useful operational baselines, but they do not directly optimise the delayed interaction between competing movements. In the selected case study, North–South and East–West movements compete for green time while stochastic arrivals change the queue configuration over the peak period. The problem is therefore not simply to classify which direction is busy; it is to repeatedly select a control action whose consequences propagate into future queue lengths and spillback risk. Demand-responsive signal-control strategies such as OPAC also motivate the broader adaptive-control framing [10].

1.2 Motivation

The cost of a signal decision is delayed. A keep decision can discharge the currently active approach and appear locally beneficial, while allowing the opposing queue to reach capacity several steps later. Conversely, switching too early introduces lost service and unnecessary phase changes. Demand can also reverse within the same peak window, so a split that is appropriate early in an episode may become inappropriate later. The controller therefore needs to trade immediate discharge against future congestion rather than optimise each detector snapshot independently. This is the sequential structure that motivates the MDP formulation and the use of temporal-difference learning.

1.3 Why Reinforcement Learning?

Reinforcement learning provides a natural closed-loop formulation. The agent is the signal controller; the environment is the simulated isolated intersection; the state is the discretised queue vector, current phase and green timer; and the action is keep or switch. After the action, stochastic arrivals and discharge produce a next state and a scalar reward. The reward penalises total queue occupancy, blocked arrivals caused by spillback and unnecessary switching. The long-term objective is the discounted return rather than instantaneous queue minimisation, using a discount factor close to one so that delayed congestion outcomes

are properly valued. Because the transition model is not supplied analytically to the agent, sampled temporal-difference updates are used to learn action values from interaction with the simulator. The TD formulation follows the standard reinforcement-learning framework [1].

1.4 Selected RL Algorithm

This case study selects **Q-learning** as the tabular temporal-difference control method for the isolated intersection problem. Q-learning is off-policy and model-free: it learns action values directly from simulated interaction without requiring an analytic transition model, and its learned target is already aligned with the greedy policy that would be used for deployment. Other tabular methods such as SARSA (on-policy), Expected SARSA (policy-averaged target) and Dyna-Q (model-based planning) are recognised as closely related approaches and remain natural candidates for empirical comparison in subsequent experimental work; however, the present case study focuses, as required, on justifying a single suitable algorithm for the formulated problem.

1.5 Objectives

- Formulate isolated adaptive signal control as a finite, discrete MDP with an explicit safety-related minimum-green constraint.
- Define the RL environment – environment, agent, state variables, action space and reward function – with justification for each design choice.
- Characterise the resulting RL problem along standard dimensions such as episodicity, stationarity, stochasticity, observability and objective horizon.
- Select and justify a suitable tabular RL algorithm (Q-learning) for the formulated problem.
- Propose a Safe RL innovation (a hard action shield) appropriate for eventual real-time deployment.

1.6 Report Organization

Section 2 reviews related RL traffic-signal studies and identifies the project gap. Section 3 defines the real-time case study and its sequential decision structure (Problem Definition). Section 4 specifies the RL Environment Formulation, including state, actions, reward and episode settings. Section 5 characterises the RL problem. Section 6 selects and justifies the RL algorithm. Section 7 explains the RL workflow. Section 8 presents the Safe RL innovation and future extension, followed by the conclusion and references.

2. Related Work

The selected case study is positioned within a progression from early multi-agent and tabular traffic-signal RL to function approximation and deep, coordinated methods. The studies below are the principal traffic-control works informing this case study, supplemented by the Safe RL survey that motivates the proposed safety layer. They show why adaptive control is promising while also motivating the compact, transparent tabular formulation used here.

Study	Problem/Domain	RL Approach	Key Finding	Limitation
Wiering [3]	City-scale traffic-light control	Multi-agent RL with communicated information	RL controllers can outperform non-adaptable controllers, particularly as traffic load and inter-intersection interaction increase.	Car/value-function representation and coordination are more complex than the isolated tabular MDP used here.

Study	Problem/Domain	RL Approach	Key Finding	Limitation
Prashanth & Bhatnagar [4]	Traffic signal control with high-dimensional state	RL with function approximation	State-action features make RL applicable when full-state tabulation becomes computationally difficult.	Function approximation is unnecessary for the compact 6,480-state coursework environment and reduces transparency.
van der Pol & Oliehoek [5]	Coordinated traffic-light control	Deep Q-learning with coordination	Coordination and a redesigned reward can reduce travel times and address multi-light control.	Designed for coordinated multi-agent settings rather than the isolated two-phase intersection studied here.
Wei et al. [6]	Intelligent traffic-light control using real traffic data	Deep RL (IntelliLight)	Evaluated on a large real-traffic dataset; examines learned policies as well as rewards.	Requires a larger state representation and data pipeline than the compact tabular simulator used for this project.
Chen et al. [7]	Large-scale multi-intersection traffic networks	Decentralized deep RL with pressure-based coordination	Well-designed local rewards can provide implicit coordination and scale to very large networks.	The scale and deep representation are beyond the scope of the single tabular algorithm used here.

2.1 Project Gap

Prior traffic-signal RL research spans multi-agent coordination, function approximation and deep RL, often targeting networks larger than the isolated junction used here. The present case study deliberately addresses a narrower methodological gap: a single, well-justified tabular TD-control method (Q-learning) is formulated on one fixed MDP so that the state representation, action space, reward design and safety constraint can be defined clearly before any empirical training is undertaken. The same compact formulation also makes the safety extension explicit. The case study therefore provides a controlled bridge between textbook Units 4–5 and more complex deep or multi-agent traffic systems, and lays the groundwork on which a broader empirical comparison of related TD methods could later be built.

3. Problem Definition (Component A)

3.1 Real-Time Problem

The case study considers an isolated four-leg intersection with two signal phases. North and South share phase 0, while East and West share phase 1. Each approach is represented by a one-dimensional occupancy stack with five discrete queue bins. At every simulation tick, the controller must decide whether to keep the current phase or request a switch. Vehicle arrivals are stochastic and their directional rates drift during the episode: North–South demand is heavier early, while East–West becomes heavier later. A green approach can discharge one vehicle per step unless a small random stall occurs. When an arrival finds a full approach, the event is treated as blocked demand or spillback. The practical objective is to reduce waiting and overflow while avoiding excessive phase changes.

3.2 Why the Problem Is Sequential and Real-Time

At time t , the queue observed by the agent is partly the result of earlier phase decisions. The action at t changes which approaches can discharge, which changes the queue vector at $t+1$. New stochastic arrivals then alter the next decision state. A keep decision can therefore create a future spillback event even when the current queue sum is acceptable, while a switch can incur an immediate cost but prevent a larger downstream

delay. This delayed credit assignment is exactly the setting for TD bootstrapping: the value of an action includes an estimate of the future value of the resulting state. The decision also has to be made continuously, tick by tick, as the intersection operates – making the problem inherently real-time rather than a one-shot classification or planning exercise.

3.3 Agent, Decisions and Long-Term Objective

The **agent** is a signal controller that maintains an action-value table $Q(s,a)$. It observes only the current discretised queues, active phase and green timer. It does not route vehicles or control vehicle speeds. The explicit **decision** at every step is binary: action 0 keeps the current phase and action 1 requests a switch to the complementary phase. A switch is ineffective until a minimum-green duration has been satisfied. A legal switch flips the phase and resets the timer. The **long-term objective** is to maximise the expected discounted return $G = \sum \gamma^k R_{t+k+1}$, using a discount factor close to one. Because rewards are negative costs, better policies produce returns closer to zero by maintaining shorter queues, reducing blocked arrivals and limiting unnecessary switching.

3.4 Why Rule-Based Systems Are Insufficient

Fixed-time plans depend on assumed average flows and cannot reallocate green when demand changes within a peak. Gap-based actuation reacts to vehicles currently present but is still largely myopic. Simple queue-difference rules require hand-tuned thresholds and do not naturally incorporate the minimum-green timer as part of the decision state. A model-predictive controller could look ahead, but it requires a sufficiently accurate predictive model at every tick and is computationally heavier than a learned tabular lookup. RL is therefore attractive for the adaptive component because it learns a closed-loop mapping from occupancy patterns to actions while retaining a compact real-time policy. Classical fixed-time signal-setting work provides the baseline context for these approaches [2].

4. RL Environment Formulation (Component B)

4.1 Environment

The environment is the custom **TrafficSignalEnv** simulator, which generates Poisson vehicle arrivals, applies phase-dependent discharge and random stalls, enforces the minimum-green constraint, and returns the resulting reward. The environment is fully observed by construction, although real detector systems would introduce partial observability and measurement error. It is dynamic, in that arrival rates drift within an episode, and stochastic, in that discharge and arrivals are randomised.

4.2 Agent

The agent is the signal controller described in Section 3.3. It interacts with the environment purely through the keep/switch action and receives the discretised state and scalar reward at every tick. It maintains a tabular action-value function $Q(s,a)$ that is updated from experience rather than from an explicit model of arrivals or discharge.

4.3 State Variables (with Justification)

The state is $s = (q_N, q_S, q_E, q_W, \phi, t_{\text{green}})$, encoded as one integer for the tabular method. Each approach has six possible occupancy levels (0–5), the phase has two values and the timer has five discrete levels, giving $|S| = 6^4 \times 2 \times 5 = 6,480$ states. The representation is intentionally compact so that the chosen tabular method can use a single, fully enumerable Q-table.

State Variable	Description	Type/Range	Justification
q_N	North approach occupancy bin	0–5	Captures congestion on North while keeping the table finite.
q_S	South approach occupancy bin	0–5	Captures opposing North–South demand and remaining discharge pressure.
q_E	East approach occupancy bin	0–5	Captures East queue growth and spillback risk.
q_W	West approach occupancy bin	0–5	Captures opposing East–West demand and saturation.
ϕ	Active signal phase	{0, 1}	The same queue vector has different valid actions depending on whether NS or EW has green.
t_green	Current green duration level	0–MIN_GREEN N	Determines whether switching is currently legal and makes the constrained decision Markov.

4.4 Action Space

The action space is discrete with two actions: $a = 0$, keep the current phase; and $a = 1$, request a switch. There is no continuous green-time action because the case study is designed around tabular Units 4–5 methods. If $t_green < MIN_GREEN$, the environment ignores a switch request and treats it as keep while the timer increments. If the minimum green has been satisfied, a switch changes the active phase and resets the timer. This environment-level lock prevents premature switching and is the first hard safety constraint in the simulator.

4.5 Reward Function

At every step the simulator returns $R = -\sum_i q_i - 8 \cdot n_{\text{blocked}} - 0.35 \cdot 1_{\text{switched}}$. Queue occupancy represents ordinary waiting cost, blocked arrivals receive a substantially larger penalty because they indicate spillback, and a legal switch has a small cost that discourages unrealistic phase chatter. There is no separate throughput bonus because discharge already reduces future queue occupancy. Returns closer to zero therefore indicate better performance under the stated cost design.

Situation	Outcome	Reward	Reason
Ordinary queue occupancy	Vehicles remain waiting	$-\sum q_i$	Makes every additional queued vehicle costly and encourages sustained queue reduction.
Blocked arrival / spillback	Full approach cannot accept an arrival	–8 per blocked arrival	Makes overflow substantially worse than ordinary delay.
Legal phase switch	Phase changes after minimum green	–0.35	Represents lost time and discourages unnecessary switching.
Illegal early switch request	Minimum-green lock keeps phase unchanged	No extra switch penalty; action is ineffective	The environment enforces the hard constraint rather than allowing an unsafe transition.
Improved discharge	Green approach serves vehicles	Indirect improvement in future return	Discharge lowers later queue occupancy, so no separate throughput bonus is needed.

4.6 Episode Definition

Each episode represents one simulated peak window of $T = 120$ discrete steps. The initial condition contains a small random residual queue and a random initial phase so that the environment is not restricted to empty intersections. The episode terminates when the horizon is reached, and its discounted return can then be recorded. The field problem would be closer to continuing operation, but a finite episode is appropriate for the coursework formulation and keeps the definition of "episode" well specified for the RL problem characterisation that follows.

5. RL Problem Characteristics (Component C)

The formulated MDP can be characterised along the standard dimensions used to describe reinforcement learning problems, as follows.

Episodic / Continuing. Episodic for the formulated task: each peak window has $T = 120$ steps and a terminal time. Field operation of a real intersection would instead be continuing, with no natural terminal state.

Static / Dynamic. Dynamic: arrival rates drift during an episode, including a directional demand flip from NS-heavy to EW-heavy, so the effective demand pattern is not fixed over the course of an episode.

Deterministic / Stochastic. Stochastic: arrivals follow a Poisson process and discharge can experience random stalls, so the same state-action pair (s, a) can produce different next states and rewards on different occasions.

Fully / Partially Observable. Fully observed in simulation: exact discrete queues, phase and timer are available to the agent. A field deployment would be closer to a POMDP because real detectors are imperfect and introduce measurement noise.

State and Action Space Type. Both the state space and the action space are discrete and finite: 6,480 encoded states and two actions (keep, switch), with the action space additionally constrained by the minimum-green lock.

Sequential Decision-Making. The problem is sequential by construction: the phase decision at time t changes which approaches discharge, which alters the queue configuration – and hence the decision problem – at time $t+1$, as discussed in Section 3.2.

Online / Offline / Simulation-Based Learning. Simulation-based, online interaction is intended: the agent's Q-values would be updated from transitions generated by interacting with TrafficSignalEnv step by step, rather than from a fixed, pre-collected offline dataset of logged field transitions.

Long-Term versus Short-Term Objective. The objective is explicitly long-term: the agent maximises the discounted sum of future rewards rather than the immediate one-step reward, so that a locally costly switch that prevents a later spillback event is preferred over a locally cheap keep decision with worse downstream consequences.

6. Selection of RL Algorithm (Component D)

6.1 Candidate Algorithms Considered

Dynamic Programming (DP) methods require a known transition model and reward function; since TrafficSignalEnv's arrival and discharge dynamics are not supplied to the agent analytically, DP is not directly applicable without first learning a model. Monte Carlo (MC) control methods only update value estimates at the end of a complete episode and can suffer from high variance under the stochastic arrival process used here, which is a poor match for a controller that should improve its behaviour continuously, tick by tick, in a real-time setting. Temporal-difference (TD) methods, by contrast, bootstrap from the current value estimates after every single step, learn directly from simulated interaction without a model, and are well suited to the

compact, finite state–action representation defined in Section 4. Within the TD family, Q-learning, SARSA, Expected SARSA and Dyna-Q are all plausible candidates; this case study selects one of them, Q-learning, and justifies the choice below.

6.2 Selected Algorithm: Q-Learning

Q-learning is an off-policy, one-step TD control method. Its update rule is $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$, where α is a small step-size (learning-rate) constant and γ is a discount factor close to one. The target assumes the best next action regardless of which action an exploratory behaviour policy (such as ϵ -greedy) actually selects. During interaction, the agent would follow an exploratory policy to visit a broad range of queue configurations, while the value function it learns is already aligned with the greedy policy $\pi(s) = \operatorname{argmax}_a Q(s, a)$ that would be deployed operationally.

6.3 Justification of Suitability

- **Model-free and off-policy:** the transition model of TrafficSignalEnv (Poisson arrivals, stalls, demand drift) is not given analytically, and Q-learning learns purely from sampled transitions without needing to estimate this model.
- **Deployment-aligned target:** because Q-learning bootstraps from the maximum next-state value, the learned Q-table already represents the greedy, deployment-style policy, which matches the intended real-time use of the controller.
- **Compatibility with the compact state–action representation:** the 6,480-state, two-action MDP defined in Section 4 is small enough for an explicit tabular Q-table, so no function approximation is required and the algorithm remains fully transparent.
- **Natural extensibility:** Q-learning is the base update used by several related extensions, including Dyna-Q (model-based planning) and Double Q-learning (bias correction), so selecting it as the base method keeps the door open for such extensions in future empirical work.

7. RL Workflow (Component E)

The closed loop below summarises how the environment, state, agent, action, reward and learning update interact at every simulation tick under the selected Q-learning algorithm: Environment $\rightarrow$ State $\rightarrow$ Agent $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ Next State $\rightarrow$ Learning (Q-update) $\rightarrow$ Policy, with the updated policy in turn shaping how the agent acts on the environment going forward.

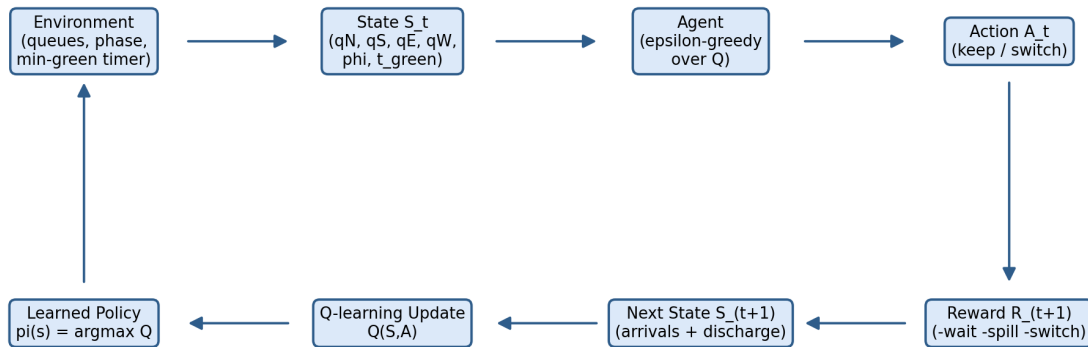


Figure 1. RL workflow for the selected case study under Q-learning.

At each tick the agent observes the current state $S_t = (q_N, q_S, q_E, q_W, \phi, t_{\text{green}})$ and selects an action A_t (keep or switch) using an exploratory policy over Q . The environment applies the action – subject to the minimum-green lock – generates new Poisson arrivals and discharge, and returns a reward R_{t+1} together with the next state S_{t+1} . The Q-learning update then adjusts $Q(S_t, A_t)$ toward the observed reward plus the discounted value of the best action in S_{t+1} , and the resulting improved policy $\pi(s) = \arg\max_a Q(s, a)$ governs how future actions are chosen, closing the loop back to the environment.

8. Innovation Component (Component F)

The selected innovation is **Safe Reinforcement Learning with a hard action shield**. The simulator already enforces minimum green as an environment-level lock; the proposed extension makes safety a separate decision layer that can reject an unsafe learned action before it reaches the signal controller. The safe action set $A_{\text{safe}}(s)$ can require minimum green and yellow/all-red clearance, maximum red limits, a spillback veto when an approach is saturated, and a pedestrian-call lock during an active walk interval. The agent would then maximise Q only over actions in $A_{\text{safe}}(s)$. This is stronger than reward shaping alone because a finite negative penalty can still, in principle, be outweighed by a large expected throughput gain, whereas a hard shield removes unsafe actions from consideration entirely. This follows the Safe RL distinction between modifying the objective and modifying the exploration or action process [8].

Candidate innovation directions considered for this case study include:

- Multi-Agent RL
- Digital Twin + RL
- **Safe RL – selected innovation**
- Explainable RL
- Hierarchical RL
- Predictive RL
- Offline RL
- Deep RL

The shield improves the system by separating optimisation from safety enforcement. The learning agent can still adapt phase allocation to changing demand, but the final action is restricted to the safe set. Hard constraints such as pedestrian clearance, maximum red and imminent spillback cannot be traded away for a higher expected return. A longer-term constrained cost could additionally be tracked with a second value table and a Lagrangian or PID-Lagrangian multiplier when the requirement is a frequency constraint rather than a single-state prohibition. This creates a deployment-oriented architecture in which learning proposes an action, the shield validates it, and only then does the signal controller receive the phase request.

9. Conclusion

This case study formulates adaptive control of an isolated four-approach intersection as a discrete sequential decision problem. The environment contains 6,480 states formed from four queue bins, the active phase and the minimum-green timer, with two constrained actions: keep and switch. Stochastic Poisson arrivals, random discharge stalls and a within-episode directional demand flip make the environment dynamic and stochastic, and it is characterised in Section 5 as episodic, dynamic, stochastic, fully observable in simulation, discrete in both state and action space, sequential, suited to simulation-based online learning, and driven by a long-term discounted objective. Q-learning is selected and justified as the most suitable tabular TD-control algorithm for this formulation because it is model-free, off-policy and produces a policy that is directly deployable. Safe RL shielding is proposed as the innovation component so that hard operational constraints are enforced independently of the learned reward optimisation before any field consideration.

9.1 Key Outcomes of This Case Study

- The isolated intersection is represented as a compact 6,480-state MDP with a binary keep/switch action.
- The RL problem is characterised as episodic, dynamic, stochastic, fully observable in simulation, discrete, sequential, simulation-based, and long-term-objective driven.
- Q-learning is selected and justified as the suitable tabular algorithm for this formulation.
- Safety constraints are proposed to be enforced as a hard action-shield layer rather than relying only on reward penalties.

9.2 Future Scope

Building on this formulation, future work could implement and empirically train Q-learning on the TrafficSignalEnv simulator, calibrate the simulator to real detector traces, add yellow/all-red timing and pedestrian phases, model turning movements and downstream occupancy, and evaluate the learned policy under multiple random seeds. A digital twin could support simulation-to-field validation, while Deep RL could handle the larger state spaces created by multiple lanes and intersections. A multi-agent corridor formulation could coordinate neighbouring signals, but the Safe RL shield should remain a prerequisite for any deployment-oriented extension. Deep-RL approaches are a natural continuation when the state representation becomes too large for tabulation [9].

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] F. V. Webster, “Traffic signal settings,” Road Research Technical Paper No. 39, Her Majesty’s Stationery Office, London, U.K., 1958.
- [3] M. Wiering, “Multi-agent reinforcement learning for traffic light control,” in *Proc. ICML*, 2000, pp. 1151–1158.

- [4] L. A. Prashanth and S. Bhatnagar, "Reinforcement learning with function approximation for traffic signal control," *IEEE Trans. Intell. Transp. Syst.*, vol. 12, no. 2, pp. 412–421, Jun. 2011.
- [5] E. van der Pol and F. A. Oliehoek, "Coordinated deep reinforcement learners for traffic light control," in *Proc. Learning, Inference and Control of Multi-Agent Systems (at NIPS 2016)*, Barcelona, Spain, 2016.
- [6] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. KDD*, 2018, pp. 2496–2505.
- [7] C. Chen, H. Wei, N. Xu, G. Zheng, M. Yang, Y. Xiong, K. Xu, and Z. Li, "Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control," in *Proc. AAAI*, vol. 34, no. 4, pp. 3414–3421, 2020.
- [8] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *J. Mach. Learn. Res.*, vol. 16, pp. 1437–1480, 2015.
- [9] A. Haydari and Y. Yılmaz, "Deep reinforcement learning for intelligent transportation systems: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 1, pp. 11–32, Jan. 2022.
- [10] N. H. Gartner, "OPAC: A demand-responsive strategy for traffic signal control," *Transp. Res. Rec.*, no. 906, pp. 75–81, 1983.